

Deep Research Agent

Agentic RAG 自主深度研究系统

Agent 开发工程师面试准备材料

技术栈: LangGraph + FastAPI + Vue 3 + ChromaDB + BM25 + SSE

源代码: github.com/956501819/deep-research-agent

生成日期: 2026-05-16

一、项目概述

Deep Research Agent 是一个基于 Agentic RAG 的自主深度研究系统。与传统 RAG 不同，它不是一个简单的「检索-回答」流程——Agent 会主动拆解复杂问题、自主选择最优检索策略、评估检索结果质量、在质量不达标时自我纠正并重试，最终生成带有内联引用和溯源的结构化研究报告。

项目完整覆盖了 AI Agent 开发的核心技术栈：LangGraph 编排引擎、多 Provider LLM 封装、混合检索（向量+BM25+RRF融合）、检索质量 Critique、查询改写与重试控制、SSE 流式推送、以及 Vue 3 工程化前端。

核心数据

- Agent 核心模块: 27 个 Python 文件 (planner / retrieval / critique / synthesis)
- 后端 API: 4 个路由模块 (research / quick-search / documents / settings) ，15+ 端点
- 前端: 34 个 Vue 3 / TypeScript 文件，4 个完整功能页面
- 提交记录: 47 次渐进式 commit，从 MVP 到完整工程化落地

与传统 RAG 的关键区别

维度	传统 RAG	Deep Research Agent
检索次数	1 次	最多 3 次（自适应重试）
检索策略	固定向量检索	Agent 自主选择 semantic/keyword/hybrid
质量判断	无	LLM 双维度评分（相关性+完整性）
查询改写	无	3 级升级（broaden → switch → rephrase）
输出	单次回答	结构化报告 + 内联引用 + 溯源
Agent 框架	无 / 黑盒	LangGraph StateGraph 显式状态机

二、技术栈全景

以下按架构分层列出项目使用的所有核心技术，以及每项技术在面试中可以展开讨论的要点。

层级	技术选型	面试讨论要点
Agent 编排	LangGraph StateGraph	显式状态流转+条件路由，非黑盒AgentExecutor
LLM 接入	SiliconFlow / Qwen / OpenAI	Factory 模式统一封装，多Provider无感切换
向量存储	ChromaDB (Persistent)	嵌入式部署，余弦相似度，可切换 Milvus
关键词检索	BM25 (rank-bm25)	互补语义检索，中文分词适配
混合检索	RRF (k=60)	稠密向量 + 稀疏BM25 融合重排序
Embedding	BGE-large-zh-v1.5	1024维，本地/API双模式，批量优化
后端框架	FastAPI + SSE	异步+流式，12+ REST端点，CORS中间件
前端框架	Vue 3 + Vite + TypeScript	Composition API, Pinia, Naive UI, Tailwind
状态管理	Pinia (research/chat/documents/setti	4个Store，替代Streamlit session_state

SSE 客户端	原生 EventSource	无额外依赖，自动重连，事件分发到Pinia
文档处理	PyMuPDF + python-docx	PDF/Word/MD/TXT 四格式支持
配置管理	pydantic-settings + .env	类型安全+环境变量+热重载

三、Agent 架构详解

3.1 LangGraph 状态流转

Agent 使用 LangGraph StateGraph 显式定义了 4 个节点和 1 个条件路由。与 LangChain 的 AgentExecutor（黑盒）不同，每一步的状态变化都是显式的、可观测的。

```
用户问题
→ Decomposition Node: LLM 拆解为 2-5 个子问题，每个标注检索策略
→ Retrieval Node: 策略路由 (semantic / keyword / hybrid) + 查询改写
→ Critique Node: 双维度评分 (相关性 + 完整性)，决定 pass/fail
→ Conditional Edge (should_retry):
  [PASS] pass → next step or Synthesis
  [FAIL] fail + can retry → back to Retrieval (改写查询、切换策略)
  [FAIL] fail + exhausted → 标记低置信度，继续下一步
→ Synthesis Node: 多源聚合 + 冲突检测 + LLM 生成 + 引用标注
```

3.2 四个节点的职责

Decomposition Node（拆解）

LLM 将用户问题拆解为 2-5 个子问题，每个子问题标注最优检索策略 (semantic / keyword / hybrid)，给出策略选择理由 (rationale)。同时初始化整个研究状态：total_steps、current_step、retry_count 等。SSE 事件: research_plan_start → research_plan_chunk × N

Retrieval Node（检索）

根据当前子问题的策略标签路由到不同的检索方法：semantic 走纯向量检索、keyword 走纯 BM25、hybrid 走两者 RRF 融合。如果是重试，还会调用 QueryRewriter 改写查询——第1次 broaden（扩展）、第2次 switch（切换策略）、第3次 rephrase（改写表达）。top_k 随重试次数指数扩展。SSE 事件: retrieval_start → retrieval_result

Critique Node（评估）

LLM 对当前检索结果进行双维度评分：相关性 (relevance, 0-1) 和完整性 (completeness, 0-1)，综合得到 composite_score。评分 ≥ critique_threshold（默认 0.6）即为通过。不通过时给出 retry_suggestion，指导下一次重试的改写方向。SSE 事件: critique_start → critique_result { composite_score, relevance, completeness, passed, retry_suggestion }

Synthesis Node（合成）

所有子问题检索完成后进入合成阶段。先通过 Aggregator 对多源结果去重聚合 + 冲突检测，再通过 ReportGenerator 流式生成结构化 Markdown 报告。最后通过 CitationBuilder 为每条引用生成编号标注 [1] [2]... 并在文末附加参考资料列表。SSE 事件: synthesis_start → synthesis_chunk × N → done

3.3 自我纠错机制

这是本系统与普通 RAG 最核心的区别。当一次检索结果被评为不合格时，Agent 不会简单地接受低质量结果，而是启动一个 3 级逐步升级的重试策略：

- 第 1 次重试 (broaden): 扩展查询范围，增加同义词和相关概念
- 第 2 次重试 (switch): 切换检索策略（语义↔关键词），同时 $\text{top_k} \times 2$
- 第 3 次重试 (rephrase): 完全改写查询表达方式， $\text{top_k} \times 4$

3 次重试后若仍不达标，Agent 将当前步骤标记为 low_confidence，使用「最佳可用结果」继续，并在最终报告中标注该部分可信度较低。这种机制保证了系统不会在单点卡死，同时保持信息透明度。

四、项目亮点与面试高频问题

Q: 为什么用 LangGraph 而不是 LangChain AgentExecutor ?

A: LangChain 的 AgentExecutor 是黑盒——你无法精确控制 Agent 在每个步骤做什么决策、如何选择工具、何时停止。LangGraph StateGraph 让我显式定义了 4 个节点 + 1 个条件路由，每一步的状态变化完全可控、可观测、可调试。这在生产环境中至关重要——当 Agent 做了错误决策时，你可以精确追溯到哪个节点、什么状态下出了问题。

Q: 为什么 Critique 是独立节点而不是放在 Retrieval 里面 ?

A: 职责分离 (Separation of Concerns)。Critique 可以用 LLM 评分（当前实现），也可以替换为 Cross-Encoder 重排序模型、规则引擎、甚至人工审核——改 Critique 不影响 Retrieval，改 Retrieval 不影响 Critique。这是 Agent 架构设计的基本功：每个节点只做一件事，接口清晰，可插拔。

Q: 混合检索 vs 纯向量检索，为什么需要 BM25 ?

A: 向量检索擅长语义匹配但对于精确匹配（数字、专有名词、代码、公式）效果差。例如查询 'BERT-base 参数量 110M'，向量可能返回关于 BERT 架构的长篇描述，但 BM25 能精确命中包含 '110M' 的句子。RRF (Reciprocal Rank Fusion) 将两者的排名融合， $k=60$ 是实验验证的较优参数。

Q: 如何控制 Agent 的幻觉 ?

A: 三层控制：(1) Critique 阈值过滤——检索质量 < 0.6 触发重试而非强行回答；(2) System Prompt 约束——明确要求「检索不到就如实告知」；(3) 低置信度标记——重试耗尽后在报告中显式标注该部分的可靠性存疑。

Q: SSE vs WebSocket 为什么选 SSE ?

A: 单向推送够用——后端推送事件、前端接收展示，不需要双向通信。SSE 协议更轻量，浏览器原生 EventSource API 自带自动重连，不需要额外的 WebSocket 库。我们的 SSE 事件粒度细化到每个子步骤 (plan_chunk、retrieval_result、critique_result)，让用户实时感知 Agent 在想什么——这是 UX 的关键。

Q: 前端为什么从 Streamlit 迁移到 Vue 3 ?

A: Streamlit 开发快但局限大：(1) session_state + queue.Queue + st.rerun() 轮询模式脆弱，DOM 残留问题频发；(2) 自定义样式只能通过 unsafe_allow_html 注入 CSS；(3) 无法实现多页面路由、侧边栏导航、对话 UI 等复杂交互。Vue 3 的响应式系统天然解决状态同步，EventSource 替代轮询，Pinia 替代 session_state，组件化开发让每个功能模块独立可测试。

Q: .env 热重载如何实现？

A: PATCH /api/v1/settings → 逐行读写 .env 文件（不用 dotenv_values 解析——对特殊字符太脆弱）→ 正则匹配 KEY=VALUE → 替换或追加 → 重建 Settings 单例 → 下次 LLM/Embedding 调用即生效。API Key 字段被掩码处理（sk-***buji），掩码值不会写回 .env。

Q: 如何测试 Agent 行为？

A: 26 个 pytest 单元测试覆盖核心逻辑（decomposition / retrieval / critique / graph 流转）。Agent 运行时的每一步行为通过 SSE 事件日志完整记录——包括事件类型、时间戳、数据 payload，在生产环境中这就是天然的 audit log。

五、前端工程化

前端从 Streamlit 完全迁移至 Vue 3 + Vite + TypeScript，4 个功能页面，34 个源文件。

页面	路由	功能
深度研究	/	Agent 进度实时可视化 + Markdown 报告流式渲染 + 来源引用
快速检索	/quick-search	对话式 UI，即时问答，秒级响应
资料管理	/documents	上传/预览/删除，自动分块+嵌入+索引
系统设置	/settings	LLM/嵌入/检索配置，热重载生效

技术架构：Vue Router (4 路由) + Pinia (4 Store) + 原生 EventSource (SSE) + Naive UI + Tailwind CSS

状态管理对比 Streamlit 改进：st.session_state → Pinia Store / queue.Queue → EventSource / st.rerun() 轮询 → Vue 响应式系统

六、后端 API 设计

方法	端点	说明
POST	/api/v1/research	提交深度研究任务，返回 task_id
GET	/api/v1/research/{id}/stream	SSE 流，推送 Agent 全生命周期事件
GET	/api/v1/research/{id}	查询任务状态和最终结果
POST	/api/v1/research/{id}/cancel	取消运行中的任务
POST	/api/v1/quick-search	快速检索 + AI 摘要，同步返回，秒级响应
GET	/api/v1/documents	文件列表（含 ChromaDB 已索引文档）
POST	/api/v1/documents/upload	上传文档 (multipart)，自动分块+嵌入+索引
DELETE	/api/v1/documents/{id}	删除文档及所有关联 chunks
GET	/api/v1/settings	读取当前配置 (API Key 掩码)
PATCH	/api/v1/settings	部分更新配置，写 .env + 热重载
GET	/api/v1/settings/system-info	ChromaDB 统计 + 版本信息

SSE 事件流

事件	数据	触发时机
----	----	------

research_plan_start	{query}	开始拆解
research_plan_chunk	{index, question, strategy, rationale}	每个子问题
retrieval_start	{step, total, strategy, retry_count}	开始检索
retrieval_result	{result_count, top_score, top_prev}	检索完成
critique_start	{step}	开始评估
critique_result	{composite_score, relevance, cor	评估完成
retry_triggered	{step, count}	触发重试
synthesis_start	{total_steps}	开始合成
synthesis_chunk	{text}	报告流式片段
done	{report_length}	全部完成

七、演示流程建议

面试演示控制在 5-8 分钟，按照以下顺序逐步展示系统的核心能力：

1. 空状态展示 (30s)

展示欢迎页和三个步骤引导，介绍整体功能布局

2. 深度研究 (3min)

输入「Transformer相比LSTM有哪些优势？」→ 观察Stepper四个阶段 → 左侧面板展示拆解的子问题和检索评分 → 右侧报告流式生成 → 来源引用卡片

3. 快速检索 (1min)

切换到侧边栏「快速检索」→ 输入问题 → 秒级回复 + AI摘要 + 来源折叠

4. 资料管理 (1min)

侧边栏「资料管理」→ 上传一个PDF → 自动分块索引 → 卡片出现 → 预览和删除功能

5. 系统设置 (30s)

侧边栏「系统设置」→ 展示 LLM/嵌入/检索配置 → 说明热重载机制

八、自我介绍模板

以下是 2-3 分钟的面试自我介绍，可根据实际情况调整：

我开发了一个基于 Agentic RAG 的深度研究 Agent，核心价值是让 Agent 具备主动研究能力，而不是被动回答。技术上用 LangGraph StateGraph 做显式状态编排——定义了拆解、检索、评估、合成四个节点和一个条件路由——每一步状态都可观测可调试。检索层是混合检索，向量+BM25+RRF融合，Agent 会根据问题特征自主选择检索策略。最核心的是自我纠错机制——检索质量不达标时，Agent 自动改写查询、切换策略，最多3次逐步升级重试。后端 FastAPI + SSE 流式推送，前端 Vue 3 + TypeScript 做了完整的工程化落地——四个功能模块、34个前端文件、12+ REST端点。整个项目47次渐进式提交，从设计文档到代码都有完整记录。

九、深度追问预案

Q: 为什么不直接用 LangChain 的 `create_retrieval_chain` ?

A:

那是单次检索+回答模式。我的场景是复杂研究问题需要多步推理——拆解成子问题、每个子问题独立检索和评估、失败后重试。`create_retrieval_chain` 做不到这个闭环。

Q: ChromaDB 如果数据量大了怎么办？

A: ChromaDB 适合中小规模（10万级chunks）。更大的场景可以替换为 Milvus——我的 VectorStore 是抽象接口，切换只需改一个类。配置里已经预留了 MILVUS_ 前缀。

Q: 多轮对话怎么设计？

A: 当前每个研究任务独立。扩展方案：在 Pinia chat store 中维护对话历史，作为 research 请求的 context 参数传入后端。SSE 事件流天然支持追加。

Q: RAGAS 评估怎么集成？

A: Critique Node 目前用 LLM 评分。可以并行接入 RAGAS 指标（faithfulness、answer_relevancy、context_precision）作为客观评估维度，与 LLM 主观评分互补。

Q: 如果 LLM 挂了怎么处理？

A: 所有 LLM 调用都有 exception 捕获+SSE error 事件。前端显示明确的错误提示。多 Provider 架构（Factory 模式）可以通过配置快速切换备用 Provider。

Deep Research Agent — Agent 开发工程师面试准备材料
技术栈: LangGraph + FastAPI + Vue 3 + ChromaDB + BM25 + SSE